

UNIVERSITE D'ANGERS

Département Informatique

RAPPORT DE SOUTENANCE DE PROJET

Année universitaire 2025 – 2026

Application Web de
Constitution de Poules
Sportives

Réalisé par :

Prénom Nom
Prénom Nom

Encadrant / Tuteur :

M./Mme NOM DU TUTEUR

Date de soutenance :

JJ mois 2026

Remerciements

Je tiens tout d'abord à remercier ...

Table des matières

Remerciements	1
1 Introduction	4
1.1 Contexte général	4
1.2 Problématique	4
1.3 Objectifs du projet	4
2 État de l’art et choix technologiques	5
2.1 Panorama des outils existants	5
2.2 Choix du langage : JavaScript natif	5
2.3 Choix de la cartographie : Leaflet.js	5
2.4 Choix du format de données : CSV	5
3 Conception des algorithmes de répartition des équipes	6
3.1 Architecture générale de l’application	6
3.2 Modélisation des données	7
3.3 Données et modèle	8
3.3.1 Fichier des clubs	8
3.3.2 Fichier des équipes	8
3.4 Étapes préliminaires communes	10
3.4.1 Vérifications préalables	10
3.4.2 Définition des capacités des poules	12
3.4.3 Initialisation des poules	12
3.5 Algorithme de répartition par distance	13
3.5.1 Sélection des graines — K-Means++	13
3.5.2 Affectation des équipes restantes	14
3.5.3 Stratégie de sauvetage	16
3.5.4 Rééquilibrage des distances moyennes	17
3.6 Algorithme de répartition par niveau	19
3.6.1 Prérequis	19
3.6.2 Calcul des quotas par statuts et par poules	20

3.6.3	Distribution des équipes	20
3.6.4	Optimisation des poules en fonction des distances parcourues . .	20
4	Implémentation et défis techniques	21
4.1	Lecture et parsing du fichier CSV	21
4.2	Intégration cartographique avec Leaflet	21
4.3	Logique de swap (échange manuel d'équipes)	23
4.4	Persistance des données avec LocalStorage	23
4.5	Gestion des événements et dynamique de l'interface	23
4.6	Difficultés rencontrées et solutions apportées	23
5	Expérience utilisateur	24
5.1	Principes de design retenus	24
5.2	Le stepper multi-étapes	24
5.3	Visualisation des résultats	24
5.4	Métriques et statistiques affichées	24
6	Conclusion et perspectives	25
6.1	Bilan du projet	25
6.2	Apports personnels	25
6.3	Perspectives d'évolution	25
	Bibliographie et Webographie	26
A	Exemple de fichier CSV d'entrée	27
B	Captures d'écran de l'application	28
C	Extraits de code complémentaires	29
D	Glossaire	30

Chapitre 1

Introduction

1.1 Contexte général

Dans le cadre des compétitions sportives amateur et semi-professionnelles, ...

1.2 Problématique

La question centrale de ce projet est donc la suivante :

Comment concevoir une application web permettant d'automatiser la constitution de poules sportives en optimisant à la fois l'homogénéité de niveau et la proximité géographique des équipes ?

1.3 Objectifs du projet

Les principaux objectifs du projet sont les suivants :

1. Import et lecture de données : ...
2. Algorithme de répartition : ...
3. Visualisation cartographique : ...
4. Interface utilisateur soignée : ...
5. Édition manuelle des poules : ...
6. Métriques et statistiques : ...

Chapitre 2

État de l’art et choix technologiques

2.1 Panorama des outils existants

2.2 Choix du langage : JavaScript natif

2.3 Choix de la cartographie : Leaflet.js

Table 2.1 – Comparaison des bibliothèques de cartographie web

Critère	Leaflet	Google Maps	OpenLayers	Mapbox
Licence	Open source	Payant (quota)	Open source	Freemium
Poids	~40 Ko	Lourd	~500 Ko	Moyen
Clé API	Non	Oui	Non	Oui
Communauté	Très grande	Très grande	Moyenne	Grande
Facilité	Élevée	Élevée	Modérée	Élevée

2.4 Choix du format de données : CSV

Chapitre 3

Conception des algorithmes de répartition des équipes

3.1 Architecture générale de l'application

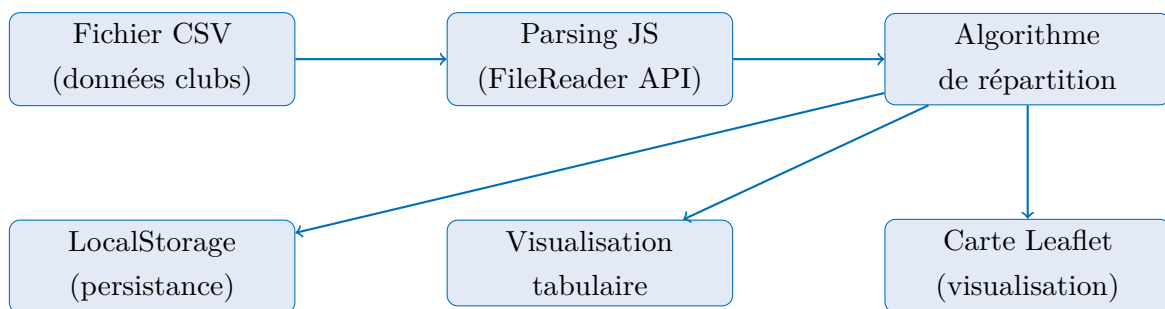


Figure 3.1 – Architecture générale du flux de données de l'application

3.2 Modélisation des données

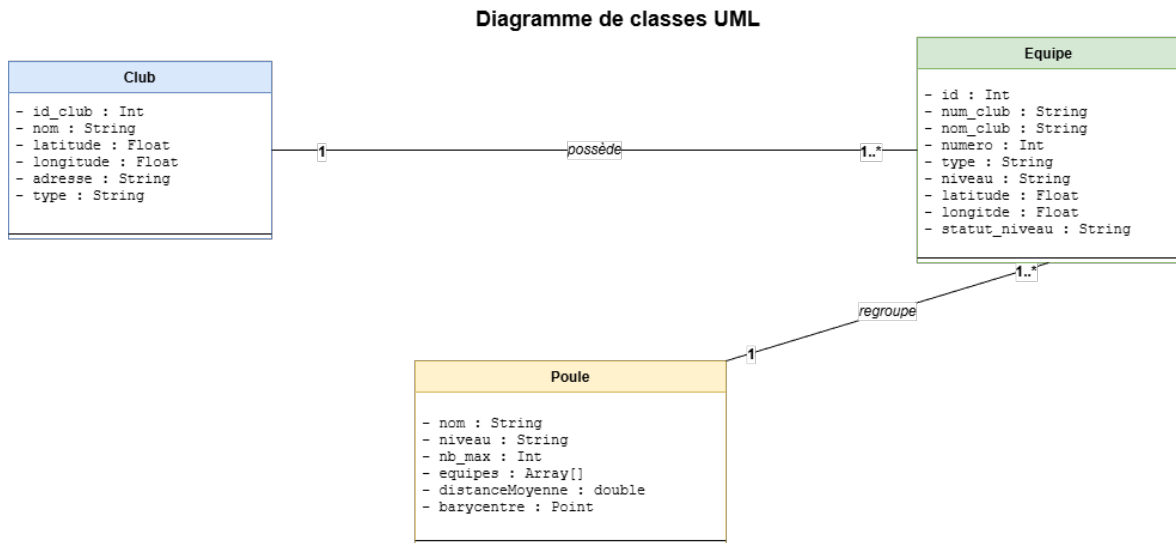


Figure 3.2 – Diagramme de classes du modèle de données

```

1
2 // Structure d'un objet "equipe" après parsing du CSV
3 const equipe = {
4   id: 1,
5   nom: "FC Exemple",
6   niveau: 3,           // Division / niveau sportif
7   latitude: 47.4784,   // Coordonnées GPS (WGS84)
8   longitude: -0.5632,
9   ville: "Angers",
10  poule: null          // Sera assigné par l'algorithme
11 };
12
13 // Structure d'une poule après répartition
14 const poule = {
15   id: "A",
16   equipes: [/* tableau d'objets equipe */],
17   barycentre: {
18     latitude: 0,
19     longitude: 0
20   },
21   distanceMoyenne: 0   // En kilomètres
22 };
  
```

Listing 3.1 – Structure d'un objet équipe en JavaScript

3.3 Données et modèle

3.3.1 Fichier des clubs

3.3.2 Fichier des équipes

Pour chaque niveau du championnat, l'organisateur dépose un fichier CSV contenant la liste des équipes participantes. Ce fichier est la principale source de données de l'application — c'est à partir de lui que les équipes sont extraites, validées et transmises à l'algorithme de génération des poules.

Structure du fichier

Le fichier est au format CSV avec une virgule comme séparateur. La première ligne contient les en-têtes des colonnes. Les colonnes exploitées par l'application sont les suivantes :

Colonne	Description	Obligatoire
CLUB_NOM	Nom du club porteur	Oui
CLUB_NUMERO	Numéro unique du club porteur	Oui
TYPE_EQUIPE	Type : <i>Club</i> ou <i>CTC</i>	Oui
CTC_NOM	Nom de la CTC	Si CTC
CTC_NUMERO	Numéro unique de la CTC	Si CTC
EQUIPE_NUMERO	Numéro de l'équipe au sein du club	Oui
NIVEAU	Niveau de compétition (ex : D1, D2)	Mode niveau
STATUT_NIVEAU	Statut : montant, descendant ou stable	Mode niveau

Table 3.1 – Colonnes exploitées dans le fichier des équipes

Les colonnes restantes du fichier (informations administratives, montants d'engagement, etc.) sont ignorées par l'application.

Robustesse de la lecture

Les index des colonnes sont déterminés dynamiquement à partir des en-têtes — l'application lit la première ligne du fichier et recherche chaque colonne par son nom. Ainsi, l'ordre des colonnes dans le fichier n'a aucune importance. Si une colonne attendue est absente, l'application signale immédiatement les colonnes manquantes à l'organisateur et interrompt le traitement.

Types d'équipes

Le fichier distingue deux types d'équipes :

- **Club** : équipe classique rattachée à un club. Son identité est définie par `CLUB_NOM` et `EQUIPE_NUMERO`. Ses coordonnées géographiques sont celles de son club.
- **CTC** (Coopération Territoriale Club) : équipe issue d'un accord entre plusieurs clubs. Son identité est définie par `CTC_NOM` et `CTC_NUMERO`. Le club porteur est identifié par `CLUB_NUMERO` — c'est lui qui fournit les coordonnées géographiques utilisées pour la génération des poules.

Vérifications appliquées

Avant de construire la liste des équipes, le traitement applique une série de vérifications ligne par ligne :

1. **Type valide** : le champ `TYPE_EQUIPE` doit correspondre à l'une des valeurs reconnues (*Club*, *CTC* ou *Coopération Territoriale Club*). Tout type inconnu interrompt immédiatement le traitement.
2. **Données complètes** : tous les champs obligatoires doivent être renseignés. Pour une CTC, `CTC_NOM` et `CTC_NUMERO` sont également requis. En mode génération par niveau, `NIVEAU` et `STATUT_NIVEAU` sont aussi obligatoires.
3. **Club connu** : le `CLUB_NUMERO` doit correspondre à un club présent dans le fichier des clubs déposé en configuration. Si le club est introuvable, l'équipe est ajoutée à une liste d'équipes inconnues. À la fin du traitement, si cette liste n'est pas vide, le traitement échoue et l'organisateur reçoit la liste complète des équipes dont le club n'a pas été trouvé.
4. **Cohérence club** : pour un même `CLUB_NUMERO`, le `CLUB_NOM` doit toujours être identique. De même, pour un même `CTC_NUMERO`, le `CTC_NOM` doit toujours être identique. Toute incohérence est signalée et interrompt le traitement.
5. **Unicité du numéro d'équipe** : deux équipes du même club et du même type ne peuvent pas avoir le même numéro. Si un doublon est détecté, le numéro est automatiquement incrémenté jusqu'à trouver un numéro disponible, et l'organisateur est averti par un message d'avertissement.

Modèle d'une équipe

À l'issue du traitement, chaque équipe est représentée par l'objet suivant :

Attribut	Description	Type
id	Identifiant unique (<i>nom-numéro</i>)	Chaîne
num_club	Numéro du club porteur	Chaîne
nom_club	Nom du club porteur	Chaîne
numero	Numéro de l'équipe au sein du club	Entier
type	<i>Club</i> ou <i>CTC</i>	Chaîne
niveau	Niveau de compétition	Chaîne
statut_niveau	Montant / Descendant / Stable	Chaîne
latitude	Latitude du club porteur	Réel
longitude	Longitude du club porteur	Réel
ctc_nom	Nom de la CTC (si CTC uniquement)	Chaîne
ctc_num	Numéro de la CTC (si CTC uniquement)	Chaîne

Table 3.2 – Attributs d'une équipe après traitement du fichier CSV

Les attributs `ctc_nom` et `ctc_num` ne sont présents que pour les équipes de type CTC — les équipes de type Club ne possèdent pas ces attributs.

3.4 Étapes préliminaires communes

3.4.1 Vérifications préalables

Avant de lancer la génération des poules, le système effectue trois vérifications sur les paramètres saisis par l'organisateur. Ces vérifications permettent de détecter immédiatement toute configuration impossible ou incohérente, et d'informer l'utilisateur avec un message d'erreur précis.

Vérification 1 — Capacité totale suffisante

La première vérification s'assure que le nombre total de places disponibles dans toutes les poules est suffisant pour accueillir toutes les équipes.

Condition :

$$nb_poules \times nb_max \geq nb_equipes$$

Si cette condition n'est pas respectée, certaines équipes ne pourront pas être placées. L'organisateur doit alors soit augmenter le nombre de poules, soit augmenter la taille maximale par poule.

Exemple : 50 équipes, 8 poules de 5 $\rightarrow$ capacité totale = 40 < 50 $\rightarrow$ **refusé**

Vérification 2 — Équilibre des tailles de poules

La deuxième vérification garantit que les poules générées seront équilibrées. On accepte qu'une poule ait au plus **un exempt** (une place vide), mais on exige qu'au moins une poule soit **complètement pleine**.

Cela revient à vérifier que le nombre d'équipes est strictement supérieur au nombre total de places si chaque poule avait une équipe de moins que le maximum.

Condition :

$$nb_equipes > nb_poules \times (nb_max - 1)$$

Si cette condition n'est pas respectée, toutes les poules auraient un exempt, ce qui signifie que l'organisateur aurait pu choisir une taille maximale inférieure dès le départ.

Exemple : 15 équipes, 3 poules de 8 $\rightarrow 3 \times 8 = 24 \geq 15 \rightarrow$ toutes les poules auraient 3 places vides $\rightarrow$ **refusé**, l'organisateur devrait choisir 3 poules de 5 directement.

Vérification 3 — Contrainte de club

La troisième vérification s'assure qu'il sera possible de répartir les équipes en respectant la règle fondamentale : **deux équipes d'un même club ne peuvent pas être dans la même poule**.

Pour que cette règle soit toujours respectée, il faut que le club ayant le plus d'équipes n'en ait pas plus que le nombre de poules disponibles. En effet, si un club a plus d'équipes que de poules, il est mathématiquement impossible de les séparer.

Condition :

$$\max_{c \in \text{clubs}} (nb_equipes(c)) \leq nb_poules$$

Exemple : un club avec 5 équipes, 4 poules $\rightarrow$ impossible de placer ces 5 équipes dans 4 poules sans conflit $\rightarrow$ **refusé**

Synthèse

Le tableau 3.3 résume les trois vérifications dans l'ordre où elles sont effectuées.

#	Condition vérifiée	Message si non respectée
1	$nb_poules \times nb_max \geq nb_equipes$	Capacité insuffisante
2	$nb_equipes > nb_poules \times (nb_max - 1)$	Trop peu d'équipes
3	$\max(\text{équipes par club}) \leq nb_poules$	Contrainte club impossible

Table 3.3 — Synthèse des vérifications préalables à la génération des poules

Ces trois vérifications sont **bloquantes** : si l'une d'elles échoue, la génération est immédiatement interrompue et un message explicite est affiché à l'organisateur.

3.4.2 Définition des capacités des poules

Avant tout placement, on détermine la capacité maximale de chaque poule. Si le nombre d'équipes n'est pas divisible exactement par le nombre de poules, certaines poules auront une équipe de moins que les autres (un exempt).

Soit n le nombre d'équipes et p le nombre de poules :

$$base = \left\lfloor \frac{n}{p} \right\rfloor \quad reste = n \bmod p$$

Les $reste$ premières poules reçoivent une capacité de $base + 1$, les autres reçoivent $base$. Pour éviter que ce soient toujours les mêmes poules qui bénéficient d'une place supplémentaire, les capacités sont ensuite **mélangées aléatoirement** grâce à l'algorithme de Fisher-Yates :

Pour i de $(p - 1)$ à 1 : choisir j aléatoirement dans $[0, i]$, puis échanger $capacites[i]$ et $capacites[j]$.

3.4.3 Initialisation des poules

Une fois les capacités définies, les poules sont initialisées comme des structures vides prêtes à accueillir les équipes. Chaque poule est représentée par un objet contenant les attributs suivants :

Attribut	Description	Valeur initiale
nom	Lettre identifiant la poule (A, B, C, ...)	A, B, C, ...
niveau	Niveau de compétition de la poule	Niveau courant
nb_max	Capacité maximale de la poule	Définie en 3.4.2
equipes	Liste des équipes affectées à la poule	[] (vide)
distance_moyenne	Distance moyenne entre les équipes de la poule	0
barycentre	Centre géographique de la poule	null

Table 3.4 – Attributs d'une poule à l'initialisation

Quelques remarques sur ces attributs :

- **Nom** : les poules sont nommées par ordre alphabétique — la première poule s'appelle A, la deuxième B, et ainsi de suite. Ce nommage est purement indicatif et facilite l'affichage dans l'interface.
- **nb_max** : chaque poule reçoit sa propre capacité issue du mélange aléatoire décrit en section 3.4.2 — certaines poules auront une capacité de $base + 1$ et d'autres une capacité de $base$.

- **equipes** : le tableau est vide à l'initialisation. La taille de ce tableau est utilisée tout au long de l'algorithme pour vérifier si une poule est pleine ($|\text{equipes}| \geq \text{nb_max}$), sans avoir recours à un compteur séparé.
- **distance_moyenne** et **barycentre** : ces deux attributs sont calculés et mis à jour dynamiquement après chaque ajout d'une équipe dans la poule. Ils valent respectivement 0 et `null` tant que la poule est vide.

3.5 Algorithme de répartition par distance

L'algorithme de génération par distance est le cœur du système. Son objectif est de répartir les équipes en poules de façon à **minimiser les distances parcourues** par les équipes tout en respectant la contrainte de club. Il se déroule en quatre phases successives.

3.5.1 Sélection des graines — K-Means++

La première étape du placement consiste à choisir p équipes "**graines**", une par poule, qui serviront de points d'ancrage géographiques. Ces graines déterminent la répartition géographique initiale des poules — un mauvais choix de graines conduit inévitablement à des poules déséquilibrées.

Pourquoi ne pas utiliser le barycentre global ?

Une approche naïve consisterait à prendre les p équipes les plus éloignées du barycentre global. Cette approche est insuffisante car ces équipes peuvent être proches entre elles. Par exemple, si la majorité des équipes se trouve au nord, le barycentre est décalé vers le nord, et les équipes les plus éloignées de ce barycentre sont toutes au sud — mais elles peuvent être très proches les unes des autres, ce qui donnerait des poules mal réparties.

La solution : K-Means++

L'algorithme K-Means++ résout ce problème en sélectionnant les graines de façon à **maximiser mutuellement leur éloignement**.

- **Graine 0** : on calcule le barycentre global de toutes les équipes (unique calcul de barycentre dans cette phase), et on choisit l'équipe la plus éloignée de ce barycentre comme point de départ.
- **Graines suivantes** : pour chaque nouvelle graine G_i , on évalue chaque équipe

candidate E avec la formule :

$$d[E] = \min(dist(E, G_0), dist(E, G_1), \dots, dist(E, G_{i-1}))$$

On choisit l'équipe dont $d[E]$ est **maximal** — c'est l'équipe la plus isolée de toutes les graines déjà placées.

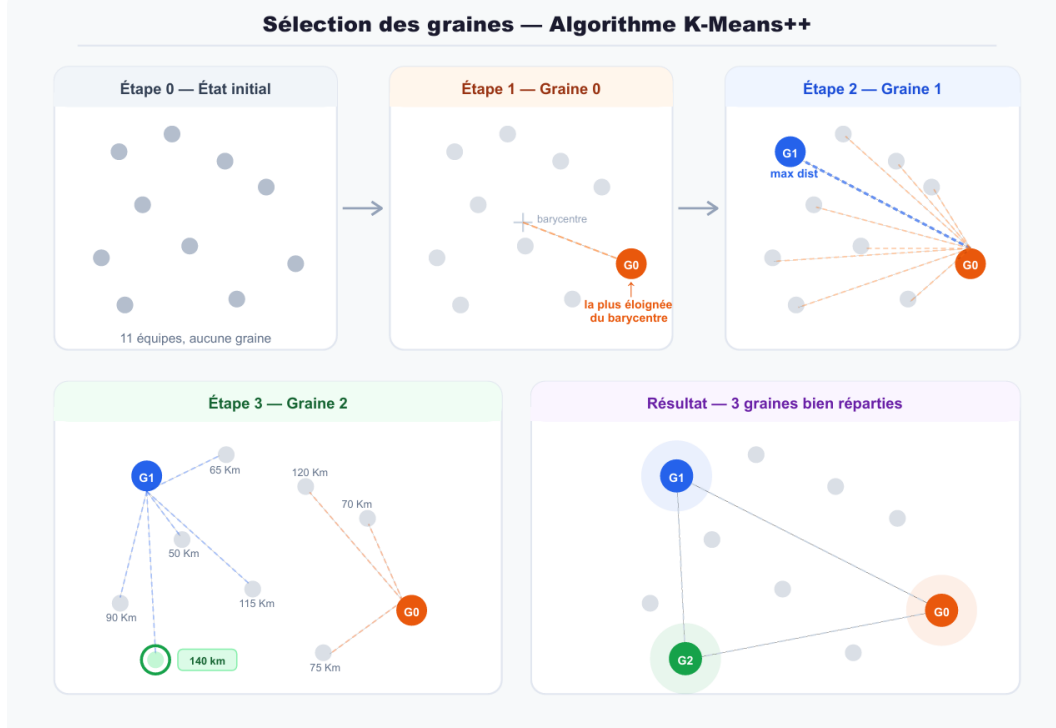


Figure 3.3 — Sélection des graines par l'algorithme K-Means++

Une fois les p graines sélectionnées, chacune est placée dans une poule distincte. Cela garantit que les poules sont initialisées dans des zones géographiques bien séparées.

“latex

3.5.2 Affectation des équipes restantes

Les équipes non sélectionnées comme graines sont ensuite affectées une par une. Elles sont d'abord **triées par ordre décroissant de leur distance au barycentre global** — les équipes les plus isolées géographiquement sont traitées en priorité, quand les poules ont encore de la flexibilité pour les accueillir.

Pour chaque équipe E à placer, on parcourt toutes les poules et on applique le processus de décision suivant :

1. **La poule est-elle pleine ?** Si $|P| \geq nb_max \rightarrow$ on passe à la suivante.
2. **Y a-t-il un conflit de club ?** Si la poule contient déjà une équipe du même club que $E \rightarrow$ on passe à la suivante.

3. **Calcul de proximité** : on évalue la compatibilité géographique de E avec la poule selon le critère retenu (voir ci-dessous).
4. On retient la poule dont le score est **minimal**.

Choix du critère de sélection de la meilleure poule

Lors de l'affectation d'une équipe E à une poule, deux critères de sélection sont envisageables. Nous les comparons ici pour justifier notre choix.

Critère 1 — Barycentre le plus proche On choisit la poule dont le barycentre géographique est le plus proche de l'équipe à placer :

$$P^* = \arg \min_{P \in \mathcal{P}} \text{dist}(E, \text{bary}(P))$$

Avantage : très rapide — un seul calcul de distance par poule candidate, soit une complexité de $O(p)$ pour placer une équipe.

Inconvénient : le barycentre est une approximation du centre géographique de la poule. Il ne reflète pas nécessairement l'impact réel de l'ajout de E sur la cohésion de la poule.

Critère 2 — Distance moyenne minimale après ajout On simule l'ajout de E dans chaque poule candidate et on retient celle dont la distance moyenne après ajout est minimale :

$$P^* = \arg \min_{P \in \mathcal{P}} \bar{d}(P \cup \{E\})$$

Avantage : mesure directement l'impact réel de l'ajout sur la cohésion géographique de la poule.

Inconvénient : ce critère présente un biais important. Considérons l'exemple suivant :

Poule	Dist. moy. avant	Dist. moy. après	Augmentation
P_A	30 km	32 km	+2 km
P_B	15 km	28 km	+13 km

Table 3.5 – Exemple illustrant le biais du critère 2

Le critère 2 choisit P_B car $28 < 32$, alors que l'ajout dans P_A ne dégrade sa cohésion que de 2 km contre 13 km pour P_B . Le critère 2 **favorise systématiquement les poules déjà denses** au détriment des poules géographiquement plus dispersées, accentuant ainsi les déséquilibres entre poules.

Critère 3 — Augmentation minimale de la distance moyenne On choisit la poule pour laquelle l'ajout de E dégrade le moins la cohésion géographique, c'est-à-dire celle dont l'augmentation de la distance moyenne est minimale :

$$P^* = \arg \min_{P \in \mathcal{P}} (\bar{d}(P \cup \{E\}) - \bar{d}(P))$$

Avantage : évite le biais du critère 2 — une poule déjà dense ne sera pas systématiquement favorisée. On mesure l'impact réel de l'ajout sur chaque poule indépendamment de son état actuel.

Inconvénient : comme le critère 2, nécessite de simuler l'ajout dans chaque poule candidate, soit une complexité de $O(p \times n)$.

Discussion et choix retenu Nos tests expérimentaux ont montré que les trois critères produisent des résultats similaires en pratique. En effet, la phase de rééquilibrage (section 3.5.4) corrige les déséquilibres introduits lors de l'affectation quel que soit le critère utilisé — le nombre d'itérations de rééquilibrage restant sensiblement le même dans les trois cas. Nous avons donc retenu le **critère 1 (barycentre le plus proche)** pour sa simplicité et son efficacité computationnelle. Notons que le barycentre de chaque poule est **recalculé et mis à jour après chaque ajout**, ce qui atténue la limite du critère 1 — le barycentre reflète à tout moment la composition réelle de la poule et non sa configuration initiale.

3.5.3 Stratégie de sauvetage

Dans certaines configurations, une équipe ne peut être placée dans aucune poule : toutes les poules sont soit pleines, soit contiennent déjà une équipe du même club. On applique alors une **stratégie de sauvetage par échange**.

Le principe est de trouver une poule P qui n'a pas de conflit avec le club de l'équipe bloquée E , d'en extraire une équipe $E_{\text{échange}}$ et de la déplacer vers une poule non pleine P_{cible} , libérant ainsi une place dans P pour E .

Pour choisir le **meilleur échange** parmi tous les échanges valides, on calcule un score combiné :

$$\text{score} = \text{dist}(E, \text{bary}_{P_{\text{après}}}) + \text{dist}(E_{\text{échange}}, \text{bary}_{P_{\text{cible}}})$$

- Le premier terme mesure la qualité de l'accueil de E dans P après le retrait de $E_{\text{échange}}$.
- Le second terme mesure la qualité de l'accueil de $E_{\text{échange}}$ dans P_{cible} .

On retient l'échange dont le score est **minimal** — les deux équipes sont ainsi placées dans les meilleures conditions géographiques possibles.

3.5.4 Rééquilibrage des distances moyennes

Même avec un algorithme glouton bien conçu, des déséquilibres peuvent subsister entre les poules après la phase d'affectation. Une poule peut se retrouver avec une distance moyenne bien supérieure aux autres, créant ainsi une inégalité sportive entre les équipes. On effectue donc une phase de **rééquilibrage par échanges améliorants**.

Principe

L'idée est simple : on cherche, parmi tous les échanges possibles entre deux équipes appartenant à deux poules différentes, celui qui **réduit le plus la somme des distances moyennes** des deux poules concernées. On applique cet échange, puis on recommence jusqu'à ce qu'aucun échange n'améliore la situation.

Critère d'évaluation

Pour chaque paire de poules (P_A, P_B) et chaque paire d'équipes $(e_A \in P_A, e_B \in P_B)$, on simule l'échange et on calcule le gain :

$$gain = (\bar{d}(P_A) + \bar{d}(P_B)) - (\bar{d}(P'_A) + \bar{d}(P'_B))$$

où :

- $\bar{d}(P)$ désigne la distance moyenne actuelle de la poule P
- P'_A désigne P_A après remplacement de e_A par e_B
- P'_B désigne P_B après remplacement de e_B par e_A

Un gain positif signifie que l'échange réduit la somme des distances moyennes des deux poules — c'est un échange améliorant. On retient l'échange dont le gain est **maximal** parmi tous les échanges valides de l'itération.

Contrainte de club

Avant d'évaluer un échange, on vérifie que la contrainte de club est respectée dans les deux sens :

- e_B ne doit pas appartenir au même club qu'une équipe déjà présente dans P_A (hors e_A qui part)
- e_A ne doit pas appartenir au même club qu'une équipe déjà présente dans P_B (hors e_B qui part)

Si l'une de ces conditions n'est pas respectée, l'échange est ignoré et on passe au suivant.

Déroulement de l'algorithme

1. Pour chaque paire de poules (P_A, P_B) avec $a < b$:
 - (a) Pour chaque paire d'équipes (e_A, e_B) :
 - i. Vérifier la contrainte de club dans les deux sens
 - ii. Calculer $\bar{d}(P'_A)$ et $\bar{d}(P'_B)$ par simulation
 - iii. Calculer le gain et mémoriser si c'est le meilleur jusqu'ici
2. Si aucun échange améliorant n'est trouvé → **convergence**, arrêt
3. Sinon, appliquer le meilleur échange et passer à l'itération suivante

Le nombre d'itérations est plafonné à **100** pour garantir la terminaison de l'algorithme dans tous les cas.

Complexité

À chaque itération, on évalue toutes les paires de poules et toutes les paires d'équipes. Grâce à la condition $a < b$ dans le code, chaque paire de poules n'est traitée qu'une seule fois, ce qui donne $\binom{p}{2}$ paires au lieu de p^2 .

Le nombre de paires de poules est :

$$\binom{p}{2} = \frac{p \times (p - 1)}{2}$$

Exemple avec $p = 4$ poules $\{A, B, C, D\}$, les paires traitées sont :

$$(A, B), (A, C), (A, D), (B, C), (B, D), (C, D) \Rightarrow \frac{4 \times 3}{2} = 6 \text{ paires}$$

Les paires symétriques $(B, A), (C, A)$, etc. ne sont jamais évaluées car l'échange (P_A, P_B) est identique à l'échange (P_B, P_A) .

Pour chaque paire de poules, on teste toutes les combinaisons d'équipes ($e_A \in P_A, e_B \in P_B$), soit n^2 combinaisons si n désigne le nombre moyen d'équipes par poule.

La complexité d'une itération est donc :

$$\binom{p}{2} \times n^2 = \frac{p(p-1)}{2} \times n^2 \sim \frac{p^2}{2} \times n^2 = O(p^2 \times n^2)$$

car en notation O , les constantes multiplicatives sont ignorées.

Dans le contexte d'un championnat de basketball régional, p et n restent faibles (typiquement $p \leq 20$ et $n \leq 10$), ce qui rend cette complexité tout à fait acceptable en pratique.

Convergence

L'algorithme converge car à chaque itération la somme des distances moyennes de toutes les poules **diminue strictement** (on n'applique un échange que si le gain est strictement positif). Cette somme étant bornée inférieurement par zéro, l'algorithme se termine nécessairement en un nombre fini d'itérations.

Formule de Haversine

```

1  /**
2   * Calcule la distance en kilomètres entre deux points GPS.
3   * @param {number} lat1 - Latitude du point 1 (degrés décimaux)
4   * @param {number} lon1 - Longitude du point 1 (degrés décimaux)
5   * @param {number} lat2 - Latitude du point 2
6   * @param {number} lon2 - Longitude du point 2
7   * @returns {number} Distance en kilomètres
8   */
9  function calculerDistance(lat1, lon1, lat2, lon2) {
10     // Implémentation de la fonction
11 }

```

Listing 3.2 – Implémentation

3.6 Algorithme de répartition par niveau

3.6.1 Prérequis

Données

La répartition des poules par niveau nécessite que toutes les équipes possèdent un attribut décrivant leur niveau au cours de la saison précédente. Cet attribut est ici nommé "**statut_niveau**" et consiste en un type énuméré de trois valeurs qui sont "-", "+" et "=". La valeur "-" traduit le fait que l'équipe soit descendue d'un niveau. La valeur "+" traduit le fait que l'équipe soit montée d'un niveau. La valeur "=" signifie que l'équipe n'a pas changé de niveau. Ainsi, les équipes de statut "-", qui proviennent du niveau supérieur sont supposées plus fortes que celles de statut "=" qui sont elle mêmes supposées plus fortes que celles de statut "+" qui proviennent d'un niveau inférieur. Les étapes suivantes s'articuleront donc autour de l'équilibrage des difficultés des poules.

Opérations

Une fois les données lues depuis le fichier correspondant, les capacités de chaque poules sont définies conformément à la section 3.4.2 et les poules sont initialisées

conformément à la section 3.4.3.

3.6.2 Calcul des quotas par statuts et par poules

L'objectif du calcul des quotas est de répartir les équipes en équilibrant le niveau de difficulté des poules. Au cours de cette étape, le nombre d'équipe de chaque statut par poule est défini en suivant une répartition équitable des équipes de ce statut. En supposant les équipes du statut x d'être au nombre de n pour un nombre p de poules, le calcul des quotas se fait suivant l'opération :

$$base = \left\lfloor \frac{n}{p} \right\rfloor \quad reste = n \mod p$$

Chaque poule contiendra au moins "base" équipes de statut x et le "reste" est réparti entre certaines poules qui reçoivent donc "base + 1" équipes de ce statut. Les poules sélectionnées pour recevoir le surplus sont obtenues par décalage circulaire du tableau contenant les quotas pour éviter que les surplus ne s'accumulent dans les premières poules.

3.6.3 Distribution des équipes

Cette étape a pour objectif de répartir les équipes dans les poules en respectant les quotas attribués à chaque poule, la contrainte d'unicité d'une équipe d'un club par poule, et la capacité des poules.

Le traitement est effectué par statut, en commençant par les équipes ayant le plus d'impact sur la difficulté de la poule, c'est-à-dire les équipes de statut "-" puis les équipes de statut "=" et pour finir les équipes de statut "+" qui sont celles ayant le moins d'impact sur la difficulté des poules les contenant. Cet ordre impose un équilibre au regard de la difficulté des poules en plaçant les équipes fortes en priorité et permet d'alléger au besoin certaines contraintes sur les équipes les plus faibles sans impacter significativement la difficulté.

Ainsi, toutes les équipes de statut " x " sont identifiées puis réparties dans les poules à tour de rôle en décrémentant le quota des poules au fur et à mesure de la distribution.

3.6.4 Optimisation des poules en fonction des distances parcourues

Chapitre 4

Implémentation et défis techniques

4.1 Lecture et parsing du fichier CSV

```
1 document.getElementById('csvInput')
2   .addEventListener('change', function (event) {
3     const file = event.target.files[0];
4     if (!file) return;
5
6     const reader = new FileReader();
7
8     reader.onload = function (e) {
9       const contenu = e.target.result;
10      const equipes = parseCSV(contenu);
11      // TODO : Expliquez ici votre fonction parseCSV()
12      afficherEquipes(equipes);
13    };
14
15    reader.onerror = function () {
16      console.error('Erreur lors de la lecture du fichier.');
```

Listing 4.1 – Lecture d'un fichier CSV avec la FileReader API

4.2 Intégration cartographique avec Leaflet

```
1 // Initialisation de la carte centrée sur la France
2 const map = L.map('map-container').setView([46.8, 2.3], 6);
3
4 // Ajout des tuiles OpenStreetMap (gratuites, sans clé API)
```

```

5 L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
6   attribution: '&copy; <a href="https://openstreetmap.org">'
7     + 'OpenStreetMap</a> contributors',
8   maxZoom: 18
9 }).addTo(map);
10
11 // Palette de couleurs pour distinguer les poules
12 const COULEURS_POULES = [
13   '#e74c3c', '#3498db', '#2ecc71', '#f39c12',
14   '#9b59b6', '#1abc9c', '#e67e22', '#34495e'
15 ];
16
17 /**
18  * Ajoute les marqueurs des équipes sur la carte.
19  * @param {Array} poules - Tableau de poules avec leurs équipes
20  */
21 function afficherMarqueursPoules(poules) {
22   poules.forEach((poule, index) => {
23     const couleur = COULEURS_POULES[index % COULEURS_POULES.length];
24
25     poule.equipes.forEach(equipe => {
26       const marqueur = L.circleMarker(
27         [equipe.latitude, equipe.longitude],
28         { color: couleur, fillColor: couleur, fillOpacity: 0.8, radius:
29           8 }
30       );
31
32       marqueur.bindPopup(
33         '<strong>${equipe.nom}</strong><br>
34         Poule : ${poule.id}<br>
35         Niveau : ${equipe.niveau}'
36       );
37
38       marqueur.addTo(map);
39     });
40   });
41 }

```

Listing 4.2 – Initialisation de la carte Leaflet et ajout de marqueurs

- 4.3 Logique de swap (échange manuel d'équipes)
- 4.4 Persistance des données avec LocalStorage
- 4.5 Gestion des événements et dynamique de l'interface
- 4.6 Difficultés rencontrées et solutions apportées

Chapitre 5

Expérience utilisateur

5.1 Principes de design retenus

5.2 Le stepper multi-étapes

5.3 Visualisation des résultats

5.4 Métriques et statistiques affichées

Chapitre 6

Conclusion et perspectives

6.1 Bilan du projet

6.2 Apports personnels

6.3 Perspectives d'évolution

En conclusion, ce projet m'a permis de ...

Bibliographie et Webographie

Documentation officielle

- MDN Web Docs – *FileReader API*. <https://developer.mozilla.org/fr/docs/Web/API/FileReader> [Consulté le JJ/MM/2026]
- MDN Web Docs – *Window.localStorage*. <https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage> [Consulté le JJ/MM/2026]
- Leaflet.js – *Documentation officielle*. <https://leafletjs.com/reference.html> [Consulté le JJ/MM/2026]
- OpenStreetMap – *Tile Usage Policy*. <https://operations.osmfoundation.org/policies/tiles/> [Consulté le JJ/MM/2026]

Articles et ressources algorithmiques

- SINNOTT, R. W. (1984). *Virtues of the Haversine*. Sky and Telescope, 68(2), 158.
- Wikipedia – *Algorithme des k-moyennes (K-means)*. <https://fr.wikipedia.org/wiki/K-moyennes> [Consulté le JJ/MM/2026]

Outils utilisés

- *Visual Studio Code* (éditeur de code). <https://code.visualstudio.com>
- *Git & GitHub* (gestion de versions). <https://github.com>

Annexe A

Exemple de fichier CSV d'entrée

```
1 nom,niveau,latitude,longitude,ville
2 FC Angers,3,47.4784,-0.5632,Angers
3 AS Saint-Laud,3,47.4851,-0.5401,Angers
4 Stade Saumurois,4,47.2606,-0.0743,Saumur
5 AS Cholet,3,47.0598,-0.8789,Cholet
6 FC Segré,4,47.6847,-0.8727,Segré
7 AS Doué,4,47.1913,-0.2792,Doué-en-Anjou
8 ÚS Longué,4,47.3762,-0.1136,Longué-Jumelles
9 FC Candé,4,47.5649,-1.0241,Candé
```

Listing A.1 – Exemple de fichier CSV d'entrée (equipes.csv)

Annexe B

Captures d'écran de l'application

Annexe C

Extraits de code complémentaires

```
1  /**
2   * Calcule le barycentre (centre de gravité) d'un ensemble d'équipes.
3   * Utilise une moyenne simple des latitudes et longitudes.
4   * Note : pour des distances intercontinentales, préférer la méthode
5   * vectorielle, mais pour des distances régionales, la moyenne suffit.
6   *
7   * @param {Array} equipes - Tableau d'objets équipe avec lat/lon
8   * @returns [{ latitude: number, longitude: number }]
9   */
10 function calculerBarycentre(equipes) {
11     if (equipes.length === 0) return { latitude: 0, longitude: 0 };
12
13     const somme = equipes.reduce(
14         (acc, equipe) => ({
15             lat: acc.lat + equipe.latitude,
16             lon: acc.lon + equipe.longitude
17         }),
18         { lat: 0, lon: 0 }
19     );
20
21     return {
22         latitude: somme.lat / equipes.length,
23         longitude: somme.lon / equipes.length
24     };
25 }
```

Listing C.1 – Calcul du barycentre géographique d'une poule

Annexe D

Glossaire

API	<i>(Application Programming Interface)</i> – Interface de programmation permettant à deux logiciels de communiquer entre eux.
Barycentre	En géographie, point moyen d'un ensemble de coordonnées GPS, utilisé pour représenter le centre géographique d'un groupe d'équipes.
CSV	<i>(Comma-Separated Values)</i> – Format de fichier texte simple dans lequel les données sont séparées par des virgules.
DOM	<i>(Document Object Model)</i> – Représentation arborescente d'une page HTML manipulable par JavaScript.
FileReader	API Web native de JavaScript permettant de lire le contenu de fichiers locaux sélectionnés par l'utilisateur.
Haversine	Formule trigonométrique permettant de calculer la distance entre deux points à la surface d'une sphère à partir de leurs coordonnées géographiques.
K-means	Algorithme de partitionnement qui regroupe des points en k clusters en minimisant la distance intra-cluster.
Leaflet	Bibliothèque JavaScript open source légère pour la création de cartes interactives dans le navigateur.
LocalStorage	API Web permettant de stocker des données clé-valeur de façon persistante dans le navigateur de l'utilisateur.
Poule	Groupe d'équipes sportives de même niveau amenées à se rencontrer lors d'une phase de compétition.
Stepper	Composant d'interface utilisateur guidant l'utilisateur à travers un processus séquentiel en plusieurs étapes distinctes.
WGS84	Système géodésique de référence mondial utilisé par le GPS, exprimant les positions en latitude/longitude décimaux.